

KubeManager

Trabalho Laboratorial nº 2 (TL2)
Laboratório de Tecnologias de Informação

EI 2024/25

Grupo 7

Martim Salvador Neves Ribeiro 2231018

Fábio Ferreira Nunes 2231014

Leiria, maio de 2026

Resumo

O presente relatório detalha o desenvolvimento e implementação do projeto KubeManager, realizado no âmbito da unidade curricular de Laboratório de Tecnologias de Informação (LTI), no decorrer do ano letivo 2025/2026. O projeto foca-se na convergência entre a orquestração de containers, através do Kubernetes, e a gestão de infraestrutura virtualizada, utilizando o Proxmox Virtual Environment.

O trabalho desenvolvido compreendeu três fases fundamentais: a análise comparativa de soluções de Kubernetes (Minikube, Kind, MicroK8S), a implementação de um cluster multi-node (1 Master e 2 Workers) em ambiente Proxmox e, por fim, a conceção de uma aplicação full-stack para a gestão centralizada destes recursos. A solução técnica baseia-se num backend robusto em .NET 9.0, que interage com as APIs oficiais do Kubernetes e Proxmox, e um frontend intuitivo desenvolvido com Tailwind CSS. As tarefas executadas incluíram a automação da clonagem de Máquinas Virtuais, a monitorização de recursos em tempo real através do Metrics Server, e a criação de funcionalidades avançadas de edição via YAML e auditoria de eventos do cluster. O relatório está estruturado em quatro partes principais: análise teórica das soluções, detalhe da infraestrutura de rede e nós, arquitetura lógica da aplicação desenvolvida e, finalmente, a documentação técnica dos pedidos de API.

Os principais contributos deste trabalho residem na simplificação da experiência de utilização do Kubernetes para administradores de sistemas, permitindo o controlo total do ciclo de vida das aplicações e da infraestrutura subjacente numa interface única e visualmente apelativa.

Palavras-chave: Kubernetes, Orquestração, Proxmox, .NET API, Containers, Monitorização.

Abstract

This detailed report details the development and implementation of the KubeManager project, carried out within the scope of the Information Technology Laboratory (LTI) curricular unit during the 2025/2026 academic year. The project focuses on the convergence between container orchestration, through Kubernetes, and the management of virtualized infrastructure, using the Proxmox Virtual Environment.

The work developed comprises three fundamental phases: the comparative analysis of Kubernetes solutions (Minikube, Kind, MicroK8S), the implementation of a multi-node cluster (1 Master and 2 Workers) in a Proxmox environment, and finally, the design of a full-stack application for the centralized management of these resources. The technical solution is based on a robust backend in .NET 9.0, which interacts with the official Kubernetes and Proxmox APIs, and an intuitive frontend developed with Tailwind CSS. The tasks performed included automating the cloning of Virtual Machines, monitoring resources in real time through Metrics Server, and creating advanced editing functionalities via YAML and cluster event audits. The report is structured in four main parts: theoretical analysis of the solutions, detailing of the network infrastructure and nodes, logical architecture of the developed application, and finally, the technical documentation of API requests.

The main contributions of this work lie in simplifying the Kubernetes user experience for system administrators, allowing total control of the application lifecycle and underlying infrastructure in a single, visually appealing interface.

Keywords: Kubernetes, Orchestration, Proxmox, .NET API, Containers, Monitoring.

Índice

Resumo.....	iii
Abstract.....	iv
Lista de Figuras.....	viii
Lista de Tabelas.....	ix
Lista de Siglas e Acrónimos	x
1. Introdução	1
2. Análise de Soluções Kubernetes	2
2.1. O que é o <i>Kubernetes</i> (Arquitetura de Control Plane e Worker Nodes)	2
2.1.1. O Control Plane (Master Node).....	2
2.1.2. Os Worker Nodes (Nós de Trabalho).....	3
2.2. Comparativa de Soluções Lightweight	4
2.2.1. Minikube.....	4
2.2.2. Kind (Kubernetes in Docker)	4
2.2.3. MicroK8s	4
2.2.4. K3s.....	5
2.3. Seleção da Solução	5
3. Implementação da Infraestrutura.....	6
3.1. Especificações Técnicas das VMs	6
3.2. Procedimento de Instalação do MicroK8s	6
3.3. Configuração do Cluster Multi-Nó	7
3.4. Otimização e Addons.....	7
3.5. Persistência de Dados na Infraestrutura	8
4. Arquitetura do Sistema e Desenvolvimento	9
4.1. Backend (ASP.NET Core Web API).....	9

4.1.1.	Processamento de Métricas e Conversão de Recursos	9
4.1.2.	Segurança e Comunicação	9
4.2.	Frontend (Interface Web).....	10
4.3.	Funcionalidades Implementadas	10
4.4.	Tratamento de Erros e Robustez	11
5.	Implementação	12
5.1.	Menu Dashboard	12
5.2.	Menu Nodes	13
5.3.	Menu Namespaces	13
5.4.	Menu Pods.....	14
5.5.	Menu Deployments.....	16
5.6.	Menu Services	17
5.7.	Menu Ingresses	18
5.8.	Menu Secrets.....	20
5.9.	Menu Eventos	21
5.10.	Menu Proxmox	22
5.11.	Menu Clusters.....	23
5.12.	Endpoints Yaml	25
5.13.	Sistema de Autenticação e Segurança	25
6.	Comparação de Experiência de Utilizador (Interface vs CLI)	27
7.	Desafios Encontrados na Implementação	28
8.	Conclusão	29
	Bibliografia.....	30

Lista de Figuras

Figura 1: Dashboard	12
Figura 2: Nodes	13
Figura 3: Listar Namespaces	14
Figura 4: Criar Namespaces	14
Figura 5: Listar e apagar pods	15
Figura 6: Criar pods	15
Figura 7: Listar e apagar deployments	16
Figura 8: Criar deployments	17
Figura 9: Criar Services	18
Figura 10: Listar e apagar Services	18
Figura 11: Listar e apagar deployments	19
Figura 12: Criar deployments	19
Figura 13: Listar e apagar secrets	20
Figura 14: Criar secrets	21
Figura 15: Mostrar eventos	21
Figura 16: Login do proxmox	23
Figura 17: Listar, controlar, clonar e apagar vms do Proxmox	23
Figura 18: Gstão de clusters	24

Lista de Tabelas

Tabela 1: Especificações Técnicas das VMs	6
Tabela 2: Endpoints da Dashboard	12
Tabela 3: Endpoints dos Nodes	13
Tabela 4: Endpoints dos Pods.....	14
Tabela 5: Endpoints de Deployments	16
Tabela 6: Endpoints de Services.....	17
Tabela 7: Endpoints de Ingresses	18
Tabela 8: Endpoints de Secrets.....	20
Tabela 9: Endpoint de Eventos	21
Tabela 10: Endpoints do menu Proxmox.....	22
Tabela 11: Endpoints do menu Clusters	24
Tabela 12: Endpoints do YAML	25

Lista de Siglas e Acrónimos

ESTG	Escola Superior de Tecnologia e Gestão
TLS	Transport Layer Security
VM	Virtual Machine

1. Introdução

O presente relatório técnico, desenvolvido no âmbito da unidade curricular de Laboratório de Tecnologias de Informação da Licenciatura em Engenharia Informática, detalha a conceção e implementação da plataforma KubeManager. Num cenário em que a transição para arquiteturas baseadas em contentores revolucionou a escalabilidade aplicacional, a gestão de ecossistemas como o Kubernetes exige ferramentas que simplifiquem a interação com as suas interfaces de programação. Para dar resposta a este desafio, o projeto foca-se na integração do Kubernetes com o Proxmox Virtual Environment, permitindo gerir, de forma centralizada, o ciclo de vida das máquinas virtuais e os recursos lógicos da camada aplicacional.

O objetivo central incide no desenvolvimento de uma aplicação completa que atue como painel de controlo unificado. O trabalho envolveu um estudo comparativo de tecnologias, a configuração de uma infraestrutura base, a criação de um motor de processamento e o desenho de uma interface interativa. Esta solução garante a monitorização dinâmica de recursos e a edição nativa de ficheiros de configuração para um controlo de gestão avançado.

O documento descreve, de forma sequencial, a seleção das ferramentas utilizadas, a implementação prática da infraestrutura, a arquitetura da aplicação desenvolvida e os resultados obtidos, culminando nas conclusões e perspetivas de trabalho futuro. Todo o projeto decorreu durante o segundo semestre do ano letivo, englobando as fases de investigação teórica, desenvolvimento de código, testes de integração e a respetiva redação técnica.

2. Análise de Soluções Kubernetes

2.1. O que é o Kubernetes (Arquitetura de Control Plane e Worker Nodes)

O *Kubernetes* (frequentemente abreviado para K8s) é uma plataforma open-source extensível e portátil, originalmente desenvolvida pela Google e atualmente mantida pela Cloud Native Computing Foundation (CNCF). A sua principal função é a gestão automatizada de cargas de trabalho e serviços baseados em *containers*, facilitando tanto a configuração declarativa como a automação de processos de implantação, escalonamento e operação de aplicações distribuídas. A arquitetura do *Kubernetes* assenta num modelo distribuído de computação do tipo *Client-Server*, onde os componentes do sistema são divididos em dois grandes blocos: o *Control Plane* (Nó Master) e os *Worker Nodes* (Nós de Trabalho). Em conjunto, estes blocos formam o que se denomina de "Cluster Kubernetes".

2.1.1. O Control Plane (Master Node)

O Control Plane é o "cérebro" do cluster. É o responsável por manter o estado desejado do sistema, tomar decisões globais sobre a alocação de recursos (escalonamento) e responder a eventos de ciclo de vida do cluster (como iniciar um novo pod quando o número de réplicas de um Deployment não é satisfeito). Num ambiente de produção, para garantir alta disponibilidade, os componentes do Control Plane são habitualmente distribuídos por múltiplas máquinas.

Os componentes centrais do Control Plane incluem:

- **Kube-apiserver:** É o componente front-end do plano de controlo. A API do Kubernetes é o ponto central de toda a comunicação do cluster, expondo uma interface RESTful através da qual os utilizadores, componentes internos e aplicações externas (como a nossa aplicação KubeManager) interagem com o sistema. Toda a alteração de estado passa por validações e autenticações neste servidor.
- **Etcd:** Trata-se de um armazém de dados chave-valor (key-value store) de alta disponibilidade e consistência, utilizado como o banco de dados principal do Kubernetes. Toda a informação sobre o estado do cluster, desde configurações de rede até secrets e definições de pods, reside exclusivamente no etcd.

- **Kube-scheduler:** É o componente responsável por vigiar pods recém-criados que ainda não foram atribuídos a nenhum nó. O escalonador analisa os requisitos de recursos (CPU, memória), políticas de afinidade de hardware, restrições e topologia da rede, e seleciona o nó mais adequado para a execução do pod.
- **Kube-controller-manager:** Executa os processos de controlo do sistema. Num cluster, existem vários controladores (ex: Node controller, Replication controller, Endpoint controller), mas estão todos compilados num único binário. O seu trabalho é monitorizar continuamente o estado atual do cluster e compará-lo com o estado desejado, efetuando as ações necessárias para corrigir discrepâncias.

2.1.2. Os Worker Nodes

Os Worker Nodes são as máquinas físicas ou virtuais onde os containers das aplicações são efetivamente executados. Se o Control Plane é a gestão, os Worker Nodes são a linha da frente de produção. O nosso projeto contemplou a configuração de dois destes nós para garantir distribuição de carga.

Cada Worker Node é composto pelos seguintes elementos fundamentais:

- **kubelet:** É o agente principal que corre em cada nó. O kubelet assegura-se de que os containers descritos nas especificações dos PodSpecs estão em execução e saudáveis. Funciona comunicando constantemente com o kube-apiserver no Control Plane para receber instruções e reportar o estado do nó e dos pods.
- **kube-proxy:** É um proxy de rede executado em cada nó, responsável pela implementação do conceito de Services do Kubernetes. O kube-proxy mantém as regras de rede no nó de hospedagem (utilizando iptables ou IPVS em Linux) que permitem a comunicação de rede para os pods, quer o tráfego venha do interior ou exterior do cluster, efetuando também um balanceamento de carga básico.
- **Container Runtime:** É o software subjacente encarregue de executar os containers. Embora o Docker tenha sido a escolha padrão histórica, o Kubernetes suporta atualmente diversas runtimes que respeitem a Container Runtime Interface (CRI), tais como o containerd ou o CRI-O. No âmbito da orquestração

moderna, o containerd tem-se afirmado como a norma devido à sua leveza e integração direta com o Kubernetes.

Esta separação clara de responsabilidades entre o plano de controlo e o plano de execução não só garante a resiliência do sistema face a falhas de hardware num nó (visto que o Control Plane o detetará e reagendará a carga de trabalho), como permite escalar a infraestrutura horizontalmente de forma simplificada, adicionando apenas novos Worker Nodes ao cluster.

2.2.Comparativa de Soluções Lightweight

No âmbito da primeira parte do trabalho laboratorial, foi realizada uma análise comparativa entre diversas soluções de Kubernetes para ambientes de desenvolvimento e produção de pequena escala.

2.2.1. Minikube

O Minikube é uma das ferramentas mais antigas e populares. Cria um cluster de um único nó dentro de uma máquina virtual (VirtualBox, KVM, etc.) ou via Docker.

- **Vantagens:** Estabilidade, suporte a múltiplos drivers, fácil de instalar.
- **Desvantagens:** Consumo elevado de recursos, dificuldade em simular clusters multi-nó reais.

2.2.2. Kind (Kubernetes in Docker)

O Kind executa nós do Kubernetes como containers Docker. É extremamente rápido e ideal para CI/CD.

- **Vantagens:** Leveza, suporte nativo a clusters multi-nó (vários containers simulando nós).
- **Desvantagens:** Persistência de dados complexa, limitações em networking avançado.

2.2.3. MicroK8s

Desenvolvido pela Canonical, o MicroK8s é uma distribuição leve, segura e fácil de instalar via 'snap'.

- **Vantagens:** Suporte nativo a clusters multi-nó reais, addons integrados (dashboard, dns, storage, metrics-server), excelente performance em hardware limitado.
- **Desvantagens:** Dependência do ecossistema Ubuntu/Snap.

2.2.4. K3s

Uma distribuição altamente otimizada pela Rancher, focada em Edge Computing e IoT.

- **Vantagens:** Binário único extremamente pequeno (<100MB), consumo mínimo de RAM.
- **Desvantagens:** Remove funcionalidades legadas que podem ser necessárias em certos contextos.

2.3. Seleção da Solução

Foi selecionado o MicroK8s devido à sua robustez e facilidade em criar um cluster com 1 Master e 2 Workers reais sobre máquinas virtuais, garantindo um ambiente que se aproxima ao máximo de um cenário de produção empresarial.

3. Implementação da Infraestrutura

A infraestrutura de suporte ao cluster foi implementada utilizando o Hipervisor Vmware Workstation Pro. Foram provisionadas três máquinas virtuais (VMs) com o sistema operativo Ubuntu 24.04 LTS.

3.1. Especificações Técnicas das VMs

Tabela 1: Especificações Técnicas das VMs

Tipo de Máquina	vCPUs	RAM	Disco	Hostname	IP
VM Master	2	2GB	50GB	kmaster	192.168.24.202
VM Worker 1	2	2GB	50GB	kworker1	192.168.24.203
VM Worker 2	2	2GB	50GB	kworker2	192.168.24.204

3.2. Procedimento de Instalação do MicroK8s

A escolha do MicroK8s permitiu uma instalação simplificada, mas extremamente robusta. Os passos executados em todos os nós foram:

Preparação do Sistema: `sudo apt update && sudo apt upgrade -y` seguido de `sudo apt install snapd -y`

Instalação via Snap: `sudo snap install microk8s --classic`

Configuração de Grupos e Permissões: `sudo usermod -a -G microk8s $USER`

As vms foram criadas partindo de clones da original após a instalação de microk8s e devido a problemas de sobreposição de IPs foi necessário correr os seguintes comandos.


```
sudo truncate -s 0 /etc/machine-id  
  
sudo rm /var/lib/dbus/machine-id  
  
sudo ln -s /etc/machine-id /var/lib/dbus/machine-id
```

Para alteração de hostname foi necessário correr o comando

```
Sudo hostnamectl set-hostname <nome>
```

E de seguida fazer Reboot

3.3. Configuração do Cluster Multi-Nó

No master foi necessário realizar as seguintes edições para garantir a comunicação. Na ausência destas alterações mesmo os comandos seguintes dando output positivo. O join do microk8s não ocorre.

É necessário editar /var/snap/microk8s/current/args/kube-apiserver e adicionar a linha

```
--bind-address=0.0.0.0
```

E também o ficheiro /var/snap/microk8s/current/certs/csr.conf.template e adicionar linha

```
IP. <numero acima do anterior> = <IP_kmaster>
```

Para transformar as instâncias isoladas num cluster foram corridos os seguintes comandos:

1. **No kmaster :** `microk8s add-node`
2. **Em cada worker (output do comando acima):**
`microk8s join <IP_DO_MASTER>:<PORTA>/<TOKEN> --worker`
3. **Verificação:** `microk8s kubectl get nodes` (confirmando os 3 nós como 'Ready').

3.4. Otimização e Addons

Após a formação do cluster, ativaram-se componentes críticos:

- **Metrics-Server:** Essencial para que a aplicação KubeManager leia o consumo de CPU/RAM em tempo real.

- **DNS:** Resolução de nomes interna entre serviços.
- **Ingress:** Exposição de serviços via NGINX Ingress Controller.

Com o seguinte comando: `microk8s enable metrics-server ingress dns`

3.5.Persistência de Dados na Infraestrutura

No que diz respeito à arquitetura de armazenamento do cluster, a implementação assente no MicroK8s utilizou o addon nativo *hostpath-storage*. Esta funcionalidade permite a criação de Persistent Volume Claims (PVCs) que mapeiam os dados diretamente para os discos rígidos virtuais das máquinas Ubuntu em execução no VMware. Esta estratégia assegura que os dados críticos (como bases de dados ou ficheiros de configuração) sobrevivem a reinícios dos Pods, garantindo que o estado da aplicação se mantém estável dentro de cada nó do cluster.

4. Arquitetura do Sistema e Desenvolvimento

A aplicação KubeManager foi desenvolvida utilizando uma arquitetura moderna e escalável, dividida num Backend em .NET e num Frontend web interativo.

4.1. Backend (ASP.NET Core Web API)

Framework: .NET 9.0 Bibliotecas: KubernetesClient (interação com a API do K8s), YamlDotNet (processamento de ficheiros YAML) e HttpClient (integração com a API REST do Proxmox).

4.1.1. Processamento de Métricas e Conversão de Recursos

O KubeManager atua como um tradutor inteligente entre os dados brutos devolvidos pela API do Kubernetes e a informação visualizada pelo utilizador. O Kubernetes reporta a utilização e os limites de recursos em formatos específicos de sistemas de orquestração (por exemplo, "100m" para milicores de CPU ou "2Gi" para Gibibytes de memória). Para contornar esta complexidade, o código do Backend implementa funções dedicadas (alojadas no `KubernetesService.cs`) que executam o *parsing* destas grandezas. Estas funções convertem matematicamente os milicores em unidades de núcleo lógicas (Cores) e transmutam capacidades como KiB e MiB para valores padronizados de Gigabytes (GB), assegurando que os relatórios gráficos apresentados no *dashboard* são imediatamente perceptíveis por qualquer utilizador.

4.1.2. Segurança e Comunicação

A segurança e a integridade da comunicação são aspetos fundamentais na plataforma KubeManager. No contexto da integração com o Proxmox, a aplicação utiliza um sistema de Tickets de Autenticação (PVEAuthCookie) e Tokens CSRF. Esta abordagem permite que, após o login inicial, a comunicação seja mantida através de identificadores temporários, minimizando a exposição contínua de credenciais. Adicionalmente, tendo em consideração que o ambiente laboratorial assenta em certificados SSL autoassinados, implementou-se uma regra de validação customizada (via `ServerCertificateCustomValidationCallback`) no `Program.cs`. Esta configuração permite que a aplicação ignore avisos de segurança de certificados não fidedignos, garantindo a fluidez das chamadas de rede entre o Backend e os servidores de infraestrutura.

4.2. Frontend (Interface Web)

Tecnologias: HTML5, JavaScript Moderno (ES6+) e Tailwind CSS. UX/UI: Modo Escuro, utilização de Modais para edição de YAML e visualização de logs, Toasts para feedback de operações e Chart.js para métricas.

4.3. Funcionalidades Implementadas

A plataforma KubeManager foi concebida para abranger desde as operações de administração até à incorporação de ferramentas avançadas, distinguindo-se por otimizar o fluxo de trabalho dos administradores de sistemas. As funcionalidades desenvolvidas agrupam-se nas seguintes capacidades:

Clonagem Dinâmica e Expansão: Facilitação da escalabilidade da infraestrutura através de um assistente de clonagem de máquinas virtuais. O sistema orquestra a criação de novas instâncias baseadas em templates ou nós existentes diretamente no Proxmox, permitindo definir novos IDs e nomes de rede de forma automatizada, acelerando o provisionamento de novos nós para o cluster.

Gestão Operacional de Recursos (CRUD): Interface completa para a gestão do ciclo de vida de recursos críticos: Namespaces, Pods, Deployments, Services e Ingresses. O sistema oferece formulários inteligentes com validação de campos, suporte a limites de hardware (CPU/RAM), gestão de variáveis de ambiente e políticas de atualização de imagem.

Editor YAML "Live Edit" com Validação: Integração com o editor profissional ACE Editor, fornecendo coloração sintática e indentação automática no browser. Todas as alterações são validadas no Backend pela biblioteca YamlDotNet antes da submissão, garantindo a integridade da configuração e prevenindo erros de sintaxe que poderiam comprometer a estabilidade do cluster.

Provisionamento de Segurança (Secrets TLS): Módulo avançado para gestão de certificados SSL/TLS. Permite a criação de Secrets de três formas: upload de ficheiros locais, colagem manual de texto ou através de um gerador criptográfico interno que cria certificados autoassinados de 2048 bits instantaneamente. Este módulo está integrado diretamente no fluxo de criação de Ingresses para facilitar a implementação de HTTPS.

Diagnóstico: Logs e Feed de Eventos: Sistema de monitorização que permite visualizar logs de contentores em tempo real e consultar o feed de eventos nativo do Kubernetes. Esta funcionalidade permite aos administradores correlacionar avisos do sistema (ex: falhas de agendamento) com o comportamento das aplicações.

Gestão de Funções (Node Roles) e Escalonamento: Capacidade de manipular as labels nativas do Kubernetes para definir funções Master/Worker nos nós e execução de operações de Patch para escalonamento horizontal de réplicas de forma dinâmica.

Utilitário de Provisionamento Remoto: Geração dinâmica de scripts Bash para automatizar a configuração de novos servidores. Este utilitário facilita a instalação da API de gestão e a extração segura de ficheiros de configuração em nós remotos, garantindo que novos servidores podem ser integrados no ecossistema KubeManager em poucos minutos.

4.4.Tratamento de Erros e Robustez

Para garantir a fiabilidade da aplicação, foi adotada uma filosofia de "Fail-fast". Logo no processo de arranque do servidor (Backend), o sistema verifica a integridade do ficheiro de configuração do Kubernetes (geralmente localizado em `~/.kube/config`). Caso este ficheiro não seja encontrado ou contenha definições inválidas, a aplicação lança imediatamente uma exceção, impedindo o arranque num estado inconsistente. Do ponto de vista do utilizador (Frontend), a plataforma implementa um sistema de notificações em ecrã dinâmico (*Toasts*). Este sistema fornece feedback visual imediato sempre que ocorre uma falha ou quando uma operação assíncrona prolongada (como a clonagem de uma Máquina Virtual no Proxmox) se encontra em processamento, mitigando a incerteza e melhorando a usabilidade geral.

5. Implementação

5.1. Menu Dashboard

Neste menu é possível observar a utilização de RAM, CPU. O número de Nodes, de Pods ativos e totais, de deployments e de namespaces.

Tabela 2: Endpoints da Dashboard

Método	Endpoint	Descrição
GET	/api/cluster	Extraí métricas agregadas do cluster ativo.

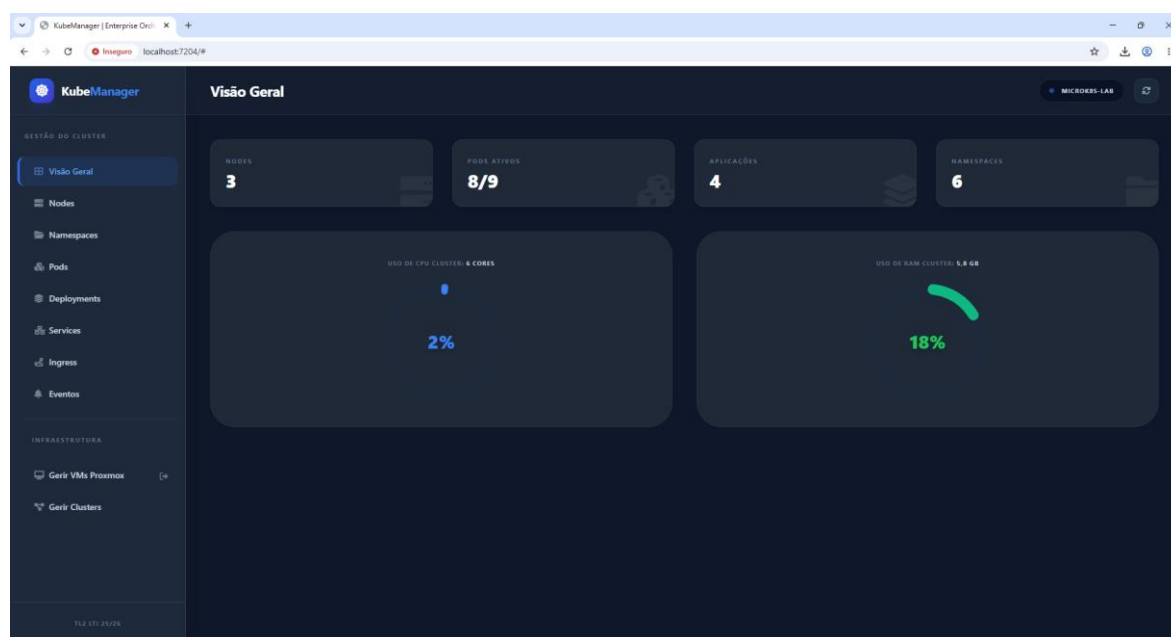


Figura 1: Dashboard

5.2.Menu Nodes

Neste menu é possível ver uma lista de todos os nodes. É possível também editar o yaml diretamente e atribuir roles aos nodes.

Tabela 3: Endpoints dos Nodes

Método	Endpoint	Descrição
GET	/api/nodes	Lista todos os nós do cluster, o seu estado e IP.
PUT	/api/nodes/{name}/role	Define ou altera a label de função (Master/Worker) do nó.

RECURSO	STATUS	ROLES	CPU	MEMORY	K8SVERSION	AÇÕES
kmaster	Ready		2	2009952K	v1.35.0	
kworker1	NotReady	worker	2	2009936K	v1.35.0	
kworker2	NotReady	worker	2	2009940K	v1.35.0	

Figura 2: Nodes

5.3.Menu Namespaces

Neste menu é possível ver uma lista de todos os namespaces, criar novos, apagar os atuais e também editá-los através da edição direta do yaml

Método	Endpoint	Descrição
GET	/api/namespaces	Lista os namespaces disponíveis.
POST	/api/namespaces	Cria um novo namespace.
DELETE	/api/namespaces/{name}	Remove um namespace e todos os recursos contidos nele.

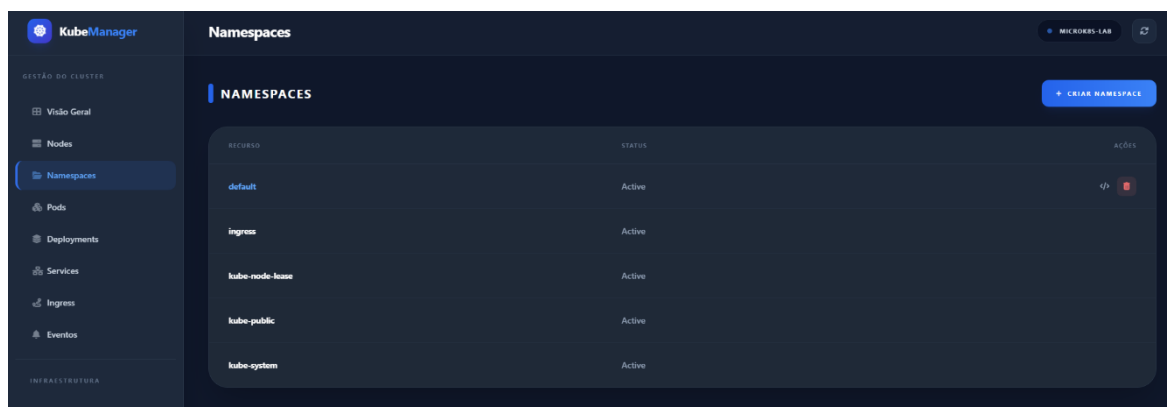


Figura 3: Listar Namespaces

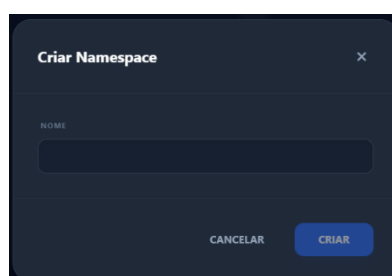


Figura 4: Criar Namespaces

5.4.Menu Pods

Neste menu é possível ver uma lista de todos os pods, criar novos, apagar os atuais e também editá-los através da edição direta do yaml.

Tabela 4: Endpoints dos Pods

Método	Endpoint	Descrição
GET	/api/pods?ns={ns}	Lista os pods filtrados por namespace.
POST	/api/pods	Criação avançada (imagem, porta, labels, variáveis de ambiente e limites).

GET	/api/pods/{name}/logs?ns={ns}	Extração de logs do contentor em tempo real.
DELETE	/api/pods/{name}?ns={ns}	Elimina o pod selecionado.

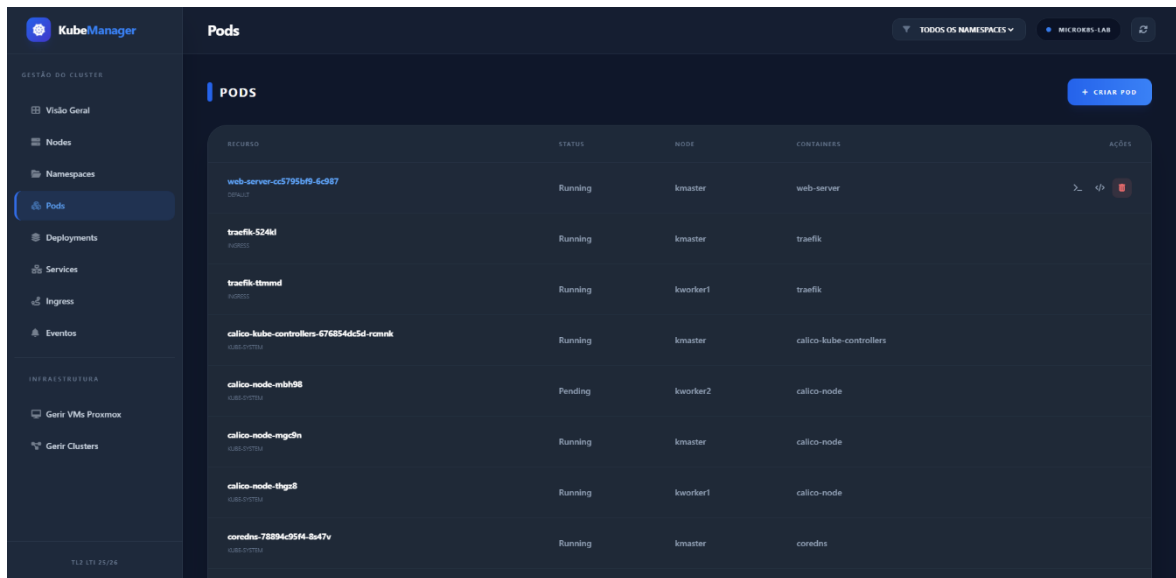


Figura 5: Listar e apagar pods

Figura 6: Criar pods

5.5.Menu Deployments

Neste menu é possível ver uma lista de todos os pods, criar novos, apagar os atuais e também editá-los através da edição direta do yaml. Também é possível alterar o número de replicas em execução

Tabela 5: Endpoints de Deployments

Método	Endpoint	Descrição
GET	/api/deployments?ns={ns}	Lista as aplicações e o estado das réplicas.
POST	/api/deployments	Criação com suporte a estratégias de atualização e pull policy.
PATCH	/api/deployments/scale	Altera o número de réplicas em execução.
DELETE	/api/deployments/{name}?ns={ns}	Elimina o deployment e os seus respetivos pods.

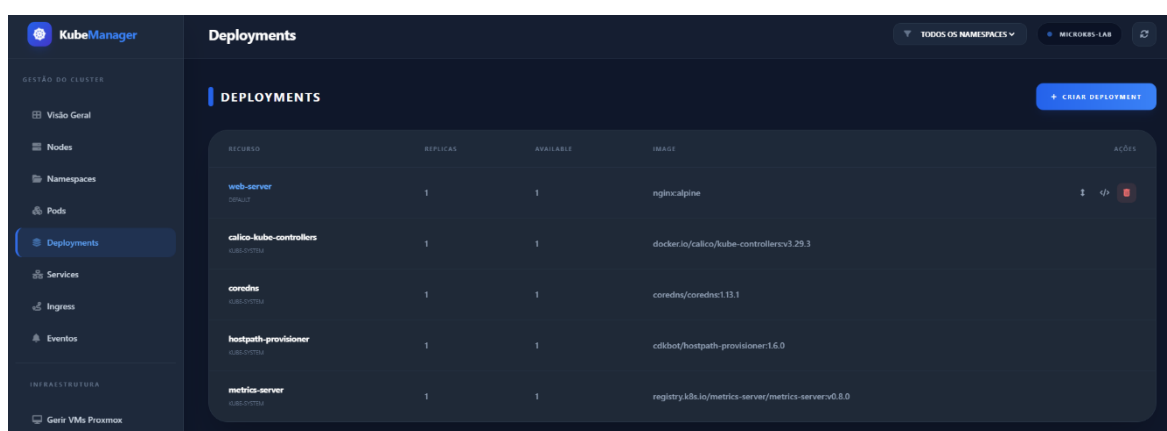


Figura 7: Listar e apagar deployments

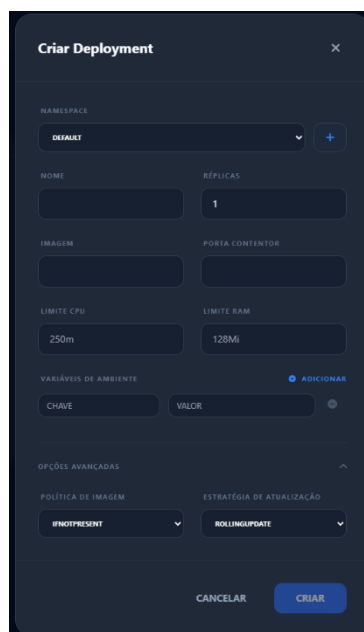


Figura 8: Criar deployments

5.6.Menu Services

Neste menu é possível a criação de Services, a listagem dos mesmos e apagá-los

Tabela 6: Endpoints de Services

Método	Endpoint	Descrição
GET	/api/services?ns={ns}	Lista os serviços e IPs internos.
POST	/api/services	Criação com mapeamento de múltiplas portas e seletores personalizados.
DELETE	/api/services/{name}?ns={ns}	Remove o serviço de rede.

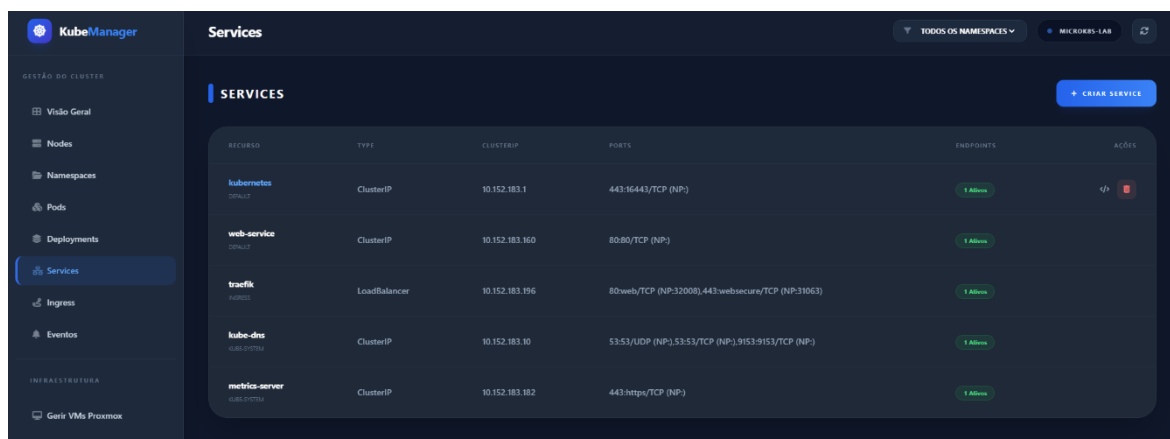


Figura 10: Listar e apagar Services

Figura 9: Criar Services

5.7.Menu Ingresses

Neste menu é possível listar os Ingresses, criar apagar e editar diretamente o yaml. Também é possível criar Secrets TLS.

Tabela 7: Endpoints de Ingresses

Método	Endpoint	Descrição
GET	/api/ingresses?ns={ns}	Lista as regras de encaminhamento HTTP.

POST	/api/ingresses	Criação com suporte a anotações e TLS (HTTPS).
DELETE	/api/ingresses/{name}?ns={ns}	Remove a regra de tráfego externo.

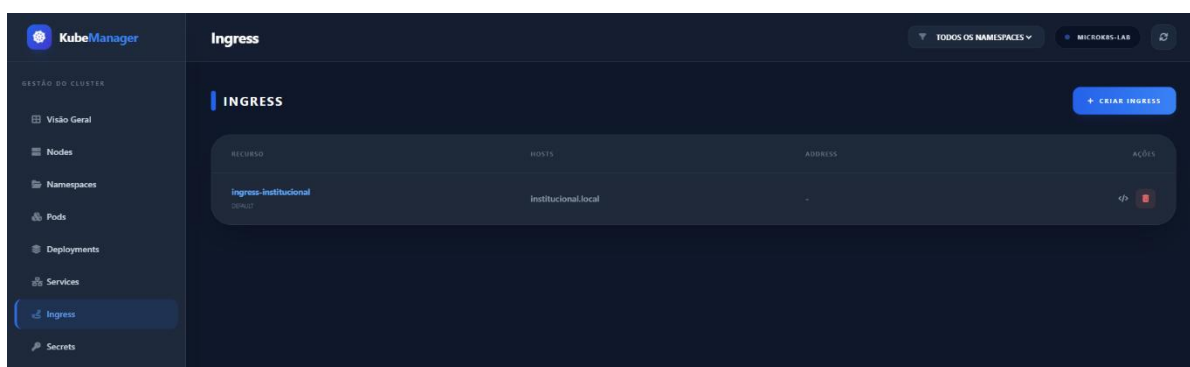


Figura 11: Listar e apagar Ingresses

Figura 12: Criar Ingresses

5.8.Menu Secrets

Neste menu é possível listar, criar e apagar secrets. A aplicação estende agora a gestão de recursos para a área de segurança criptográfica de forma avançada:

- **Gestão de TLS Secrets:** Criação simplificada de objetos do tipo `kubernetes.io/tls`, essenciais para a segurança de aplicações web e comunicações seguras.
- **Geração de Certificados Self-Signed:** O KubeManager inclui uma funcionalidade de PKI interna que permite gerar pares de Chave Privada/Certificado Público (X.509) diretamente na interface, facilitando a implementação imediata de HTTPS em serviços internos do cluster.

Tabela 8: Endpoints de Secrets

Método	Endpoint	Descrição
GET	<code>/api/k8ssecret/ns={ns}</code>	Lista certificados e segredos.
POST	<code>/api/k8ssecret</code>	Criação de segredos TLS.
POST	<code>/api/k8ssecret/generate-self-signed</code>	Gera automaticamente certificados autoassinados.
DELETE	<code>/api/k8ssecret/{name}?ns={ns}</code>	Remove o segredo de segurança.

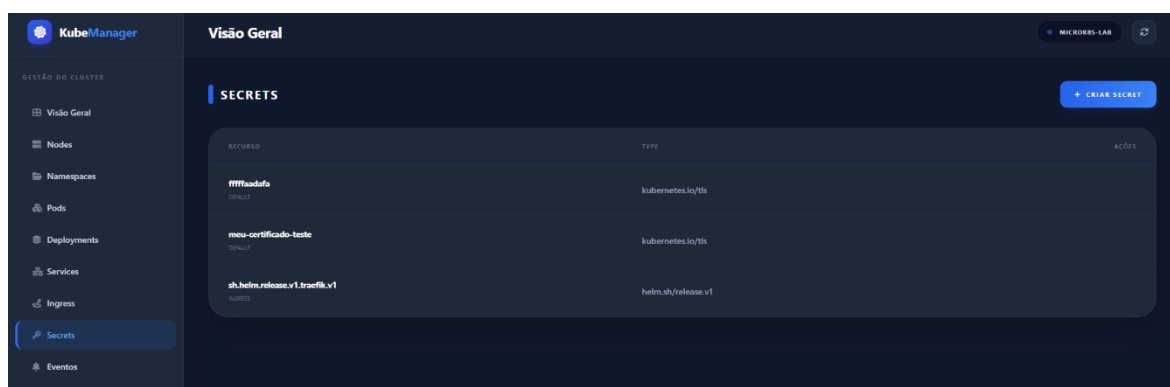


Figura 13: Listar e apagar secrets

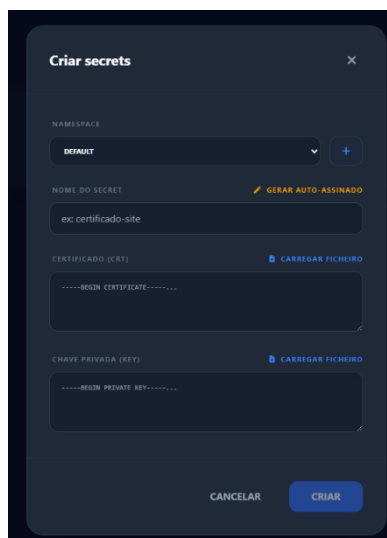


Figura 14: Criar secrets

5.9.Menu Eventos

Neste menu é possível visualizar logs de alterações em todos os componentes

Tabela 9: Endpoint de Eventos

Método	Endpoint	Descrição
GET	/api/events?ns={ns}	Obtém o feed de avisos e erros do cluster.

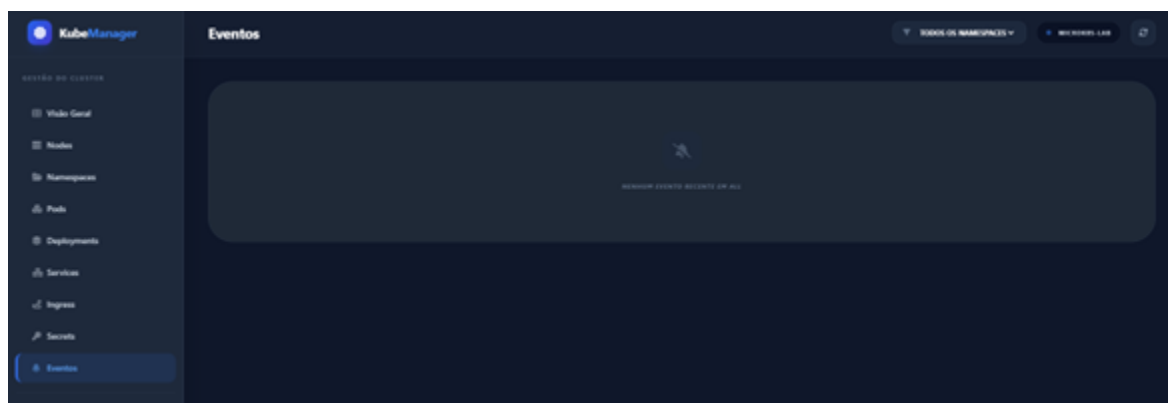


Figura 15: Mostrar eventos

5.10. Menu Proxmox

Neste menu é possível listar as vms dentro de um hypervisor Proxmox. cloná-las, ligá-las, suspender, desligar, reiniciar, parar, e retomar em caso de suspensão. Também é possível aceder à sua Shell. Mas para isso funcionar corretamente é necessário ter tido sessão iniciada na página de controlo do Proxmox.

Tabela 10: Endpoints do menu Proxmox

Método	Endpoint	Descrição
POST	/api/auth/login	Efetua a autenticação no sistema Proxmox.
GET	/api/vms	Lista todas as máquinas virtuais.
POST	/api/vms/{node}/{vmid}/action	Executa ações de energia (iniciar, parar, reiniciar).
POST	/api/vms/{node}/{vmid}/clone	Clona uma VM para a expansão da infraestrutura.
DELETE	/api/vms/{node}/{vmid}	Elimina uma máquina virtual do hipervisor.

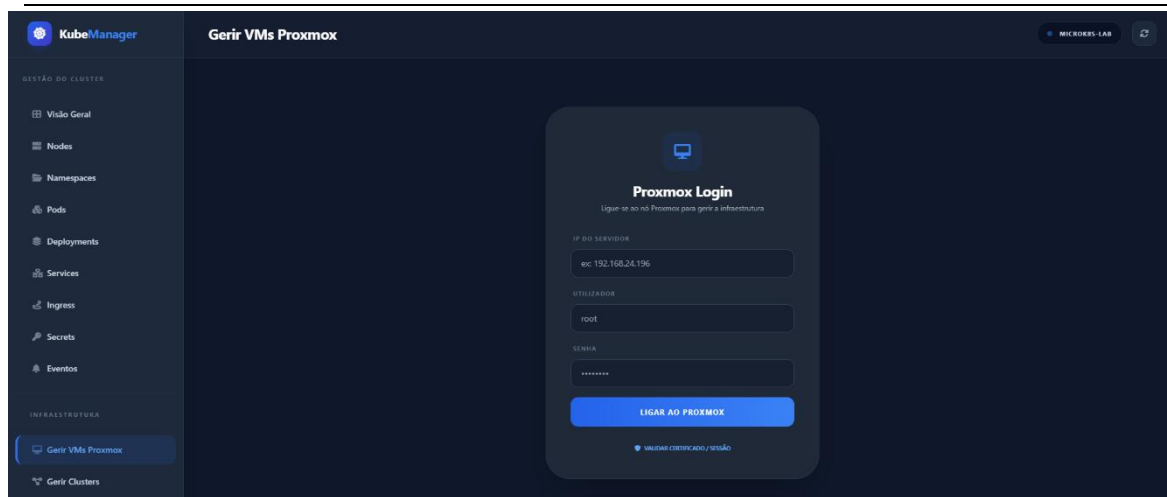


Figura 16: Login do proxmox

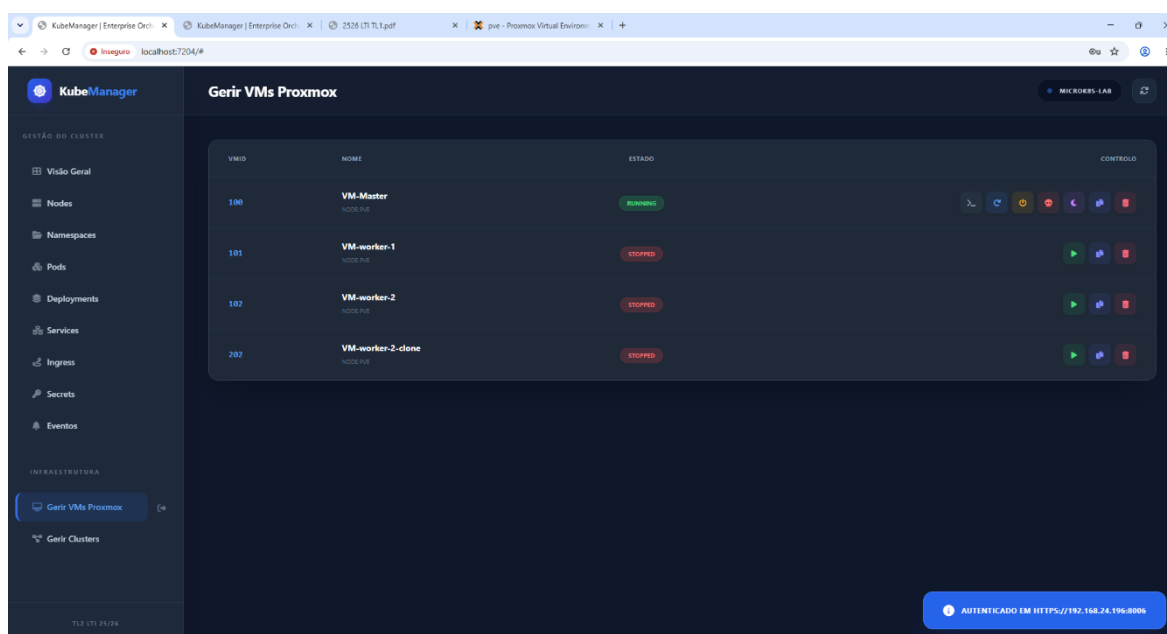


Figura 17: Listar, controlar, clonar e apagar vms do Proxmox

5.11. Menu Clusters

Uma das evoluções mais significativas da app é a capacidade de gerir múltiplos clusters a partir de uma única interface. Neste menu é possível gerir os clusters adicionados e importar novos por YAML proveniente do comando (`microk8s config`).

- **Configurações Dinâmicas:** Através do `ConfigController`, os administradores podem adicionar novas configurações de cluster (.yaml) via *upload* ou através de *fetch* remoto por IP/Porta.

- **Isolamento de Cache:** O sistema utiliza um mecanismo de cache inteligente que permite alternar entre clusters sem conflitos de configuração na visualização de dados.
- **Persistência:** Os ficheiros de configuração são guardados na pasta de configuração da API, permitindo que os clusters registados persistam entre reinícios do servidor.

Tabela 11: Endpoints do menu Clusters

Método	Endpoint	Descrição
GET	/api/config/clusters	Lista os clusters registados no painel.
POST	/api/config/clusters	Regista um novo cluster através de um ficheiro YAML.
DELETE	/api/config/clusters/{name}	Remove um cluster da gestão do painel.
GET	/api/config/fetch-yaml	Importa remotamente as configurações de novos nós.

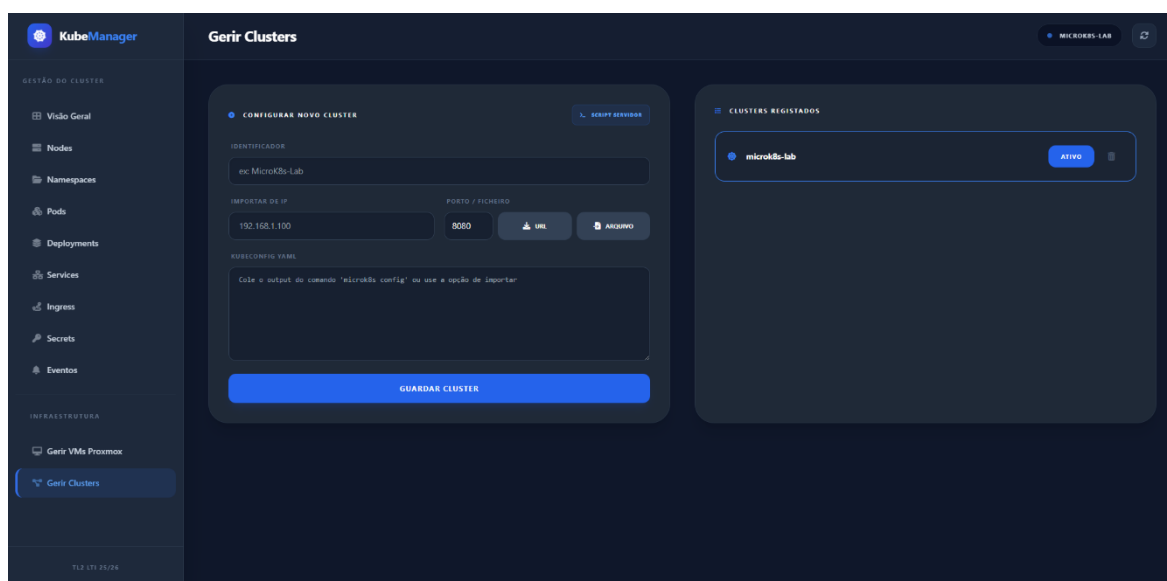


Figura 18: Gestão de clusters

Os endpoints seguintes são os utilizados sempre que ocorre edições de yaml diretamente dentro da web app.

Tabela 12: Endpoints do YAML

Método	Endpoint	Descrição
GET	/api/yaml/{resource}/{name}?ns={ns}	Obtém o YAML em bruto (bruto) para edição.
PUT	/api/yaml/{resource}/{name}?ns={ns}	Grava e valida as alterações YAML no cluster.

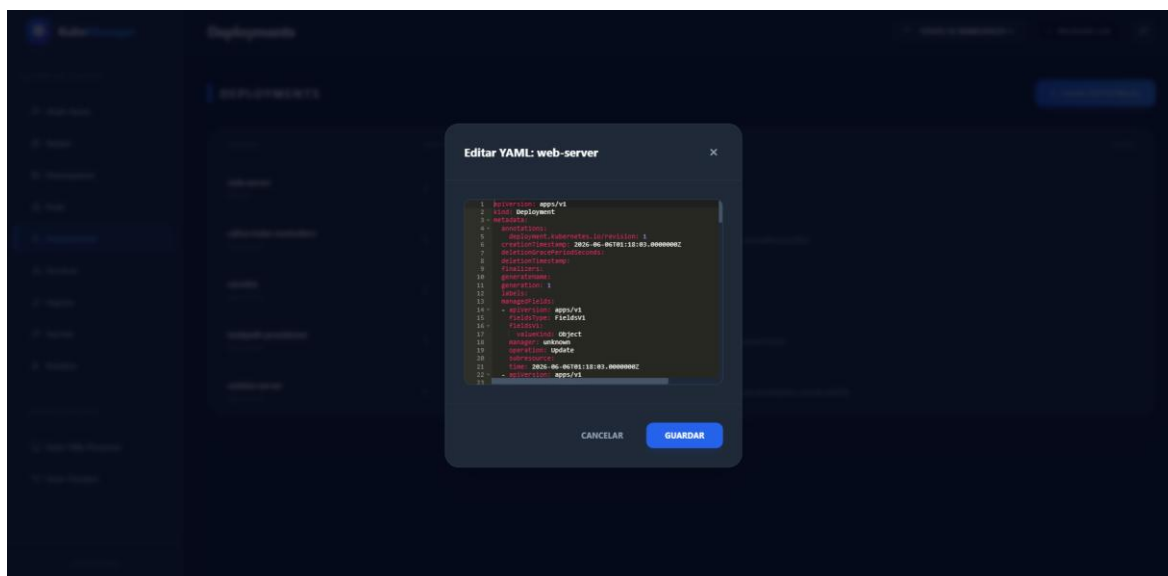


Figura 19: Editor de YAML

A aplicação implementa um módulo de autenticação robusto que permite o acesso controlado ao orquestrador.

- **Integração de Realms:** O sistema suporta diferentes domínios de autenticação (Realms), permitindo a validação de credenciais contra diferentes *backends* (por exemplo: Proxmox, PAM, AD).
- **Sessão Segura:** Após o *login*, a aplicação gere tokens de acesso que garantem a segurança e a integridade das comunicações subsequentes com as APIs do Kubernetes e do Proxmox.

6. Comparação de Experiência de Utilizador (Interface vs CLI)

A proposta de valor do KubeManager é particularmente evidente quando se compara a execução de operações rotineiras através da interface da plataforma face à utilização tradicional via linha de comandos (CLI). Tomando como exemplo a operação de escalonamento (aumento de réplicas) de um processo aplicacional (*Deployment*):

- **Via CLI (Kubectl):** O operador necessita de abrir uma sessão de terminal, autenticar-se, conhecer a sintaxe exata e executar um comando estruturado como `kubectl scale deployment/my-app --replicas=3`.
- **Via KubeManager:** O operador visualiza o *Deployment* numa lista gráfica e, com um simples clique no botão "+" da interface, submete o pedido de alteração de escala, recebendo confirmação visual imediata do sucesso da transação.

Conclui-se que a abordagem visual não só acelera a produtividade como reduz drasticamente a margem para erros humanos decorrentes de falhas de sintaxe, tornando a plataforma acessível a elementos técnicos com menor fluência nos comandos do Kubernetes.

7. Desafios Encontrados na Implementação

Durante o ciclo de desenvolvimento do projeto foram ocorrendo vários. Numa fase inicial, a orquestração foi testada com recurso ao Minikube suportado em containers. No entanto, a necessidade de simular um ambiente de produção mais fidedigno levou à transição para uma infraestrutura MicroK8s sobre o hypervisor Proxmox.

Esta mudança trouxe dificuldades acrescidas. As VMs no Proxmox demonstravam lentidão ao utilizar CPUs do tipo QEMU; no entanto, devido a problemas de nested virtualization e de Hyper-V, não foi possível alterar o tipo de CPU para o recomendado (Host). Em face disso, optámos por seguir com a implementação diretamente no VMware, dividida em três VMs distintas. Apesar desta alteração, a opção de ligar as VMs do Proxmox foi mantida, uma vez que já se encontrava implementada.

Por fim, não foi possível implementar corretamente o acesso à consola Shell das VMs incorporadas no Proxmox. Isto deveu-se ao facto de que, para garantir esse acesso, são necessários cookies de autenticação da aplicação web do Proxmox, a qual é acedida através do respetivo endereço IP.

8. Conclusão

O presente projeto culminou no desenvolvimento bem-sucedido da plataforma KubeManager, cumprindo o objetivo central de criar um painel de controlo unificado capaz de orquestrar infraestruturas baseadas em contentores e máquinas virtuais. Através da integração entre o ecossistema Kubernetes e o Proxmox Virtual Environment, foi possível demonstrar que a complexidade inerente à gestão de sistemas distribuídos pode ser significativamente mitigada através de uma interface web intuitiva e centralizada.

A implementação do cluster suportado em MicroK8s provou ser uma solução robusta e adequada para o ambiente em questão, permitindo o provisionamento de funcionalidades avançadas como a monitorização de métricas em tempo real, a persistência de dados e a gestão dinâmica de recursos aplicacionais. O desenvolvimento do motor *backend* em .NET 9.0 assegurou a comunicação segura e eficiente com as APIs das tecnologias envolvidas, enquanto o *frontend* garantiu ferramentas de administração de alto nível, tais como a edição validada de ficheiros YAML e a gestão de segurança criptográfica.

O ciclo de desenvolvimento proporcionou aprendizagens fundamentais, não apenas na área da programação *full-stack* e integração de APIs RESTful, mas também na administração de redes, arquitetura do *Control Plane* do Kubernetes e superação de obstáculos reais de infraestrutura, como os constrangimentos de *nested virtualization* que exigiram adaptações de resiliência ao longo do percurso.

Em suma, o trabalho laboratorial validou o pressuposto inicial de que uma abordagem visual otimiza a produtividade e reduz a margem de erro operacional face à utilização exclusiva de linhas de comandos, resultando num produto funcional, escalável e com forte aplicabilidade na administração moderna de sistemas.

Bibliografia

Kubernetes. "Kubernetes Documentation". Disponível em: <https://kubernetes.io/docs/>

Canonical. "MicroK8s Documentation - The lightweight Kubernetes". Disponível em: <https://microk8s.io/docs/>

Proxmox. "Proxmox VE API Viewer". Disponível em: <https://pve.proxmox.com/pve-docs/api-viewer/>

Tailwind Labs. "Tailwind CSS Documentation". Disponível em: <https://tailwindcss.com/docs>

Google. "Google Gemini". Disponível em: <https://gemini.google.com>

Google. "Gemini CLI". Disponível em: <https://geminicli.com/>

GitHub. "GitHub Copilot". Disponível em: <https://github.com/features/copilot>